

CriptoService

Predicción de Criptomonedas y Manejo de Portafolios

Integrantes:

David Nicolas Urrego Botero
durregob@unal.edu.co

Angel David Gomez Pastrana
angomezp@unal.edu.co

Andrew Nicolay Prieto Mendoza
aprietome@unal.edu.co

Jhon Edison Prieto Artunduaga
jhprieto@unal.edu.co

Profesor:

Sergio Enrique Vargas Pedraza
sevargasp@unal.edu.co

Universidad Nacional de Colombia
Facultad de Ingeniería de Sistemas e Industrial

Junio de 2026

1. Introducción

El crecimiento del mercado de las criptomonedas ha generado nuevas oportunidades de inversión, pero también ha incrementado la complejidad en la toma de decisiones debido a la alta volatilidad de los activos digitales. Los inversionistas requieren herramientas que les permitan analizar información histórica, monitorear tendencias del mercado y obtener apoyo para estimar posibles comportamientos futuros de los activos.

Con el propósito de atender esta necesidad, se desarrolló una plataforma de análisis de criptomonedas que integra herramientas de visualización, gestión de portafolios y modelos de predicción basados en técnicas de aprendizaje automático. La aplicación permite a los usuarios consultar información relevante sobre diferentes criptomonedas, administrar sus inversiones y obtener estimaciones sobre la posible dirección del mercado, facilitando así la toma de decisiones informadas.

El sistema está compuesto por una arquitectura distribuida que incluye servicios backend desarrollados en Node.js y TypeScript, una base de datos PostgreSQL para el almacenamiento de información y un servicio especializado en Python encargado de ejecutar los modelos predictivos. Esta separación de responsabilidades permite mantener una solución escalable, modular y preparada para futuras extensiones.

En términos generales, el proyecto busca ofrecer una herramienta integral para el análisis de activos digitales, combinando la gestión de inversiones con capacidades de inteligencia artificial para proporcionar información útil, actualizada y de valor para los usuarios interesados en el mercado de criptomonedas.

2. Arquitectura del Sistema

El sistema implementa una arquitectura de tipo Monolito con Servicio Auxiliar, complementada con una organización interna basada en Arquitectura en Capas.

El backend principal está desarrollado en Node.js, TypeScript y Express, concentrando la lógica de negocio relacionada con autenticación, gestión de usuarios, portafolios de inversión, criptomonedas y consumo de servicios externos. La persistencia de datos se realiza mediante PostgreSQL.

Debido a los requerimientos de análisis predictivo, se incorporó un servicio auxiliar desarrollado en Python, encargado de ejecutar modelos de Machine Learning para la generación de predicciones sobre el comportamiento de criptomonedas. La comunicación entre el backend principal y este servicio se realiza mediante solicitudes HTTP.

Internamente, el monolito se encuentra dividido en tres capas principales:

- **Controllers:** Responsables de recibir y validar las solicitudes HTTP.
- **Services:** Encargados de implementar la lógica de negocio.
- **Repositories:** Componentes que gestionan el acceso directo a la base de datos.

2.1. Diseño, Despliegue y Configuración en la Nube (AWS)

Para transformar este desarrollo en una solución de nivel empresarial, se diseñó e implementó una infraestructura en **Amazon Web Services (AWS)** enfocada en la alta disponibilidad, resiliencia operativa y segregación estricta de entornos de red mediante una **VPC (Virtual Private Cloud)** distribuida en múltiples **Zonas de Disponibilidad (AZs)**.

2.1.1. Capa de Cómputo y Sistema Operativo (Amazon EC2)

El entorno de ejecución para el servidor de aplicaciones Node.js y el microservicio analítico de Python se consolidó sobre una instancia virtual de cómputo elástico:

- **Sistema Operativo:** Se seleccionó una distribución **Ubuntu Server LTS**, elegida por su estabilidad en entornos de producción y compatibilidad nativa con los gestores de procesos globales.
- **Entorno de Red:** Ubicada dentro de una subred pública de la VPC para heredar una dirección IP pública direccionable (54.210.150.10), permitiendo que clientes externos (como Postman o el Frontend) consuman la API REST a través del puerto 3000.
- **Security Group perimetral (SG-EC2):** Actúa como firewall de red con un filtrado estricto. La regla de entrada (*Inbound Rule*) restringe el puerto 3000 a cualquier origen (0.0.0.0/0) para habilitar la operación comercial de la app, mientras que el puerto 22 para administración remota vía SSH está estrictamente limitado a la dirección IP del administrador para mitigar vectores de escaneo de vulnerabilidades.

2.1.2. Capa de Datos Transaccional Aislada (Amazon RDS)

Como respuesta directa a los hallazgos críticos de ciberseguridad relacionados con la exposición de datos, el motor relacional PostgreSQL se desplegó de forma totalmente aislada:

- **Aislamiento de Red:** La instancia de Amazon RDS se confinó exclusivamente en subredes privadas. Se desactivó de forma permanente la propiedad *Publicly Accessible* (**No**), impidiendo que el motor reciba una IP pública o enrutamiento directo desde el exterior de la red de AWS.
- **Principio de Menor Privilegio (SG-RDS):** El Security Group de la base de datos se configuró para rechazar cualquier petición externa. Solo se abrieron reglas de entrada en el puerto estandarizado 5432 cuyo origen explícito sea el identificador del **SG-EC2** de la capa de aplicación y, posteriormente, el bloque de red asignado para los procesos de ingeniería de datos.

2.1.3. Pipeline de Ingeniería de Datos y ETL Serverless (AWS Glue y S3)

Con el fin de habilitar auditorías de datos y análisis históricos masivos sin penalizar el rendimiento o bloquear la base de datos PostgreSQL de producción en la EC2, se construyó un pipeline de datos desacoplado:

1. **Conectividad Privada (AWS Glue Connection):** Al estar RDS en una subred privada, el servicio Serverless AWS Glue no posee visibilidad por defecto. Para resolverlo, se levantó una conexión de red interna de tipo JDBC denominada `Postgresql connection vf3`, la cual inyecta interfaces de red virtuales (ENIs) directamente en las subredes privadas para enlazar ambos servicios de forma interna a través de la red troncal de AWS.
2. **Descubrimiento de Esquemas (AWS Glue Crawler):** Se configuró el componente de rastreo automatizado `crawler-postgres-crypto` acoplado al rol de `IAM Role-AWS-Crypto-Glue`. Tras una ejecución de **2 minutos y 6 segundos**, el Crawler cruzó exitosamente el túnel de la VPC, autenticándose e indexando un total de **7 tablas transaccionales** dentro del AWS Glue Data Catalog de la base de datos lógica `cryptoglue-database` (mapeando tablas críticas como `postgres_public_usuario`, `crypto_prices`, `inversion` y `prediccion`).
3. **Diseño y Depuración del Job ETL (Apache Spark):** Se creó el script de extracción distribuida `job-extract-usuarios` cuyo objetivo es migrar la información histórica hacia un lago de datos en un bucket de almacenamiento masivo **Amazon S3** (`s3://criptoservice-etl-clean-data/usuarios`).

2.1.4. Bitácora de Errores y Soluciones en el Entorno Cloud

El despliegue del pipeline analítico experimentó retos críticos de configuración en la interfaz visual de AWS Glue que requirieron depuración a bajo nivel:

- **Error de Compatibilidad de Catálogo (Fallo de Compilación):** En las primeras ejecuciones del Job en Spark, el proceso fallaba devolviendo la excepción:
`Error Category: UNCLASSIFIED_ERROR; Failed Line Number: 24; An error occurred while calling o156.getCatalogSource. We don't support this connection type: POSTGRESQL.`
Causa: Se utilizó por defecto un nodo de entrada de tipo `"AWS Glue Data Catalog"`, el cual esperaba estructuras de almacenamiento analítico nativo e invalidaba la conexión directa relacional de PostgreSQL.
Solución: Se eliminó el nodo rebelde y se reemplazó por un componente de origen de base de datos relacional genérico configurado directamente mediante **JDBC**, restableciendo el mapeo de tipos de datos de Spark.
- **Bloqueo Visual de Guardado e Inconsistencia de Nombres (UI Bug):** Tras corregir el conector, la consola visual de AWS Glue mostraba de forma persistente una alerta de desconfiguración (indicador con el número rojo "1") impidiendo que el botón azul **Save** procesara los cambios.
Causa: El Crawler indexó el esquema con el prefijo compuesto proveniente del motor físico (`postgres_public_usuario`). Al intentar ingresar manualmente el nombre de la tabla como `public.usuario`, el validador de interfaz de AWS bloqueaba la compilación por discrepancia de metadatos.
Solución: Se limpió el cuadro de texto ingresando el nombre absoluto registrado por el catálogo (`postgres_public_usuario`) y se forzó un evento de enfoque en la interfaz haciendo clic

en el lienzo gris. Esto removió la alerta roja, desbloqueando con éxito el banner verde de confirmación de guardado y permitiendo la ejecución exitosa (**Run**) del pipeline de datos con salida óptima hacia Amazon S3 en formato estructurado CSV.

2.1.5. Lecciones Aprendidas e Ingeniería de Infraestructura

- **Desacoplamiento de Cargas:** La separación estricta entre la base de datos transaccional (RDS) y el almacenamiento analítico (S3 mediante Glue) es indispensable para mitigar caídas de la API causadas por la ejecución de queries pesadas de reportería.
- **Seguridad por Diseño (Security by Design):** Configurar bases de datos relacionales sin direccionamiento público en la nube reduce prácticamente a cero el escaneo malicioso automatizado que ocurre diariamente en Internet.
- **Sensibilidad de las Herramientas Visuales Cloud:** Las herramientas de ETL visuales simplifican el flujo de trabajo pero introducen restricciones de interfaz de usuario rígidas. El verdadero control del analista radica en entender la configuración subyacente del conector JDBC y las reglas de red de la VPC.

2.1.6. Arquitectura en la nube (AWS)

Debido a la alta demanda de recursos de cómputo en la capa gratuita (*Free Tier*) dentro de la región principal de N. Virginia (**us-east-1**), la cual impedía el aprovisionamiento local del motor relacional, se optó por un diseño de arquitectura híbrida inter-región altamente eficiente. Esta solución segrega las cargas de trabajo transaccionales y analíticas entre las regiones de **N. Virginia** y **Ohio** (**us-east-2**), estructurándose bajo los siguientes componentes reflejados en la Figura 1:

- **Capa de Aplicación y Acceso Frontal (N. Virginia):** El servidor de aplicaciones basado en una instancia Amazon EC2 (*Ubuntu Server*) opera en una subred pública de Virginia. Recibe las solicitudes HTTP del cliente o entornos de prueba (como Postman) expuestas a través del puerto 3000 para dar consumo a la API REST de *CriptoService*.
- **Aislamiento y Conectividad Remota de la Base de Datos (Ohio):** El motor relacional PostgreSQL de producción en Amazon RDS se confinó estrictamente dentro de una subred privada en la región de Ohio, careciendo por completo de enrutamiento o direccionamiento público hacia Internet. La persistencia de datos del backend transaccional (desde Virginia) se realiza mediante un canal directo inter-región apuntando al endpoint privado de la base de datos a través del puerto 5432.
- **Seguridad Perimetral Inter-Región:** El *Security Group* perimetral de la base de datos en Ohio se configuró bajo el principio de menor privilegio, implementando reglas de entrada estricta (*Inbound Rules*) que deniegan explícitamente cualquier petición genérica de la red, aceptando única y exclusivamente el tráfico originado desde la dirección IP pública elástica de la instancia EC2 de aplicación ubicada en Virginia.

- **Pipeline de Ingeniería de Datos Serverless (Ohio Local):** Para evitar la degradación de rendimiento de la base de datos por consultas analíticas pesadas, se acopló el servicio de ETL administrado **AWS Glue** dentro de la subred privada de Ohio. Este componente interactúa de forma local y nativa con el motor PostgreSQL mediante una conexión JDBC dedicada (`Postgresql connection vf3`), permitiendo al *Crawler* indexar el esquema transaccional en el catálogo de datos de forma segura.
- **Lago de Datos Histórico en Amazon S3:** Una vez procesado, filtrado y estructurado el flujo de información por el *Spark Job* de AWS Glue (resolviendo las inconsistencias de nombres y conectores de la tabla `postgres_public_usuario`), los datos limpios se exportan hacia el extremo derecho de la infraestructura fuera de la VPC, teniendo como destino un *bucket* global de **Amazon S3** en formato estructurado CSV para auditorías analíticas posteriores.

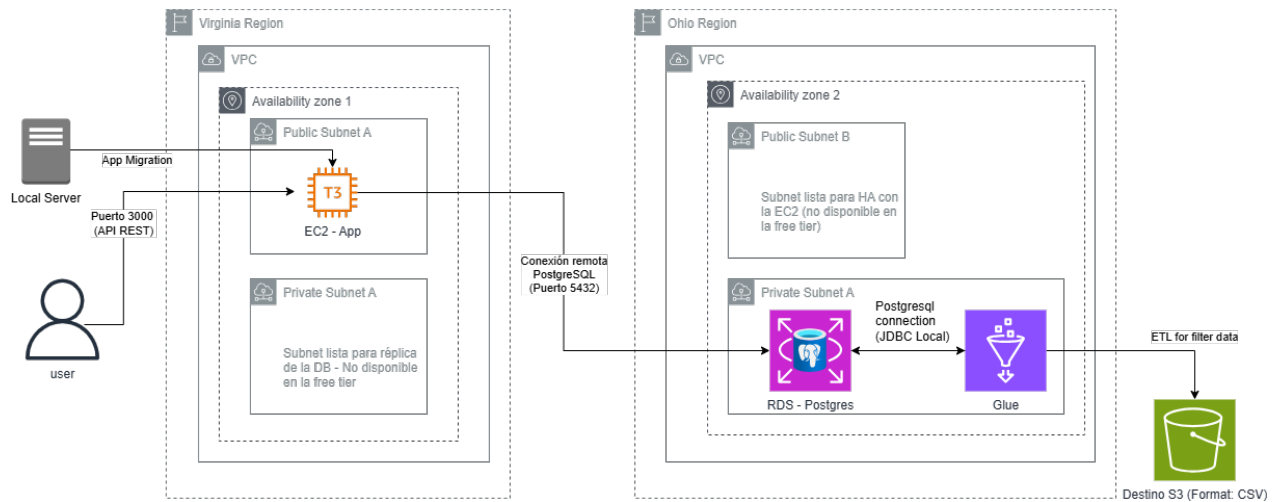


Figura 1: Diagrama Detallado de la Arquitectura Híbrida Multi-Región e Infraestructura de Datos en AWS (*CriptoService*).

3. Flujos Principales del Sistema

3.1. Flujo de Autenticación

Este flujo permite a los usuarios registrarse, iniciar sesión, utilizar autenticación multifactor (MFA) y recuperar el acceso a sus cuentas.

3.1.1. Registro de Usuario

1. El usuario envía sus datos de registro.

2. El controlador valida la información recibida.
3. El servicio de autenticación verifica que el usuario no exista previamente.
4. La contraseña es almacenada de forma segura mediante hashing.
5. La información del usuario es persistida en PostgreSQL.
6. El sistema confirma el registro exitoso.

3.1.2. Inicio de Sesión con MFA

1. El usuario envía su correo electrónico y contraseña.
2. El sistema valida las credenciales contra la información almacenada.
3. Si las credenciales son correctas, se genera un código temporal de autenticación multifactor (MFA).
4. Se solicita el código TOTP de 6 dígitos al usuario.
5. El usuario ingresa el código.
6. El sistema valida que el código sea correcto y que no haya expirado.
7. Una vez validado el MFA, se genera un token JWT.
8. El token es retornado al cliente para autorizar futuras solicitudes.

3.1.3. Acceso a Recursos Protegidos

1. El cliente envía el token JWT en cada solicitud protegida.
2. El sistema valida la firma y vigencia del token.
3. Si el token es válido, se identifica al usuario autenticado.
4. La solicitud continúa hacia los servicios correspondientes.

3.1.4. Recuperación de Contraseña

1. El usuario solicita restablecer su contraseña.
2. El sistema genera un token temporal de recuperación.
3. El token es almacenado en la base de datos junto con su fecha de expiración.
4. Se envía un correo electrónico con el enlace de recuperación.

5. El usuario accede al enlace y proporciona una nueva contraseña.
6. El sistema valida el token y verifica que no haya expirado.
7. La contraseña es actualizada de forma segura.
8. El token de recuperación es invalidado para evitar reutilización.

3.2. Flujo de Predicciones (Machine Learning)

Este flujo permite obtener predicciones sobre el comportamiento futuro de las criptomonedas.

1. El usuario solicita una predicción desde la aplicación.
2. El controlador recibe la petición y valida los parámetros.
3. El servicio de predicciones prepara la solicitud.
4. El backend envía una petición HTTP al servicio de Machine Learning en Python.
5. El servicio Python ejecuta el modelo predictivo correspondiente.
6. El resultado de la predicción es retornado al backend.
7. El backend procesa la respuesta y la envía al cliente.

Este flujo desacopla la lógica de Machine Learning del resto de la aplicación y permite aprovechar bibliotecas especializadas de Python.

3.3. Flujo de Gestión de Portafolios

Este flujo permite a los usuarios administrar sus portafolios de inversión.

1. El usuario autenticado solicita crear o consultar un portafolio.
2. El sistema valida el token JWT recibido.
3. Se verifica la existencia del usuario.
4. El servicio de portafolios ejecuta las reglas de negocio correspondientes.
5. El repositorio consulta o almacena la información en PostgreSQL.
6. El resultado es retornado al cliente.

Cada portafolio queda asociado exclusivamente al usuario propietario, garantizando el control de acceso sobre la información.

3.4. Flujo de Gestión de Inversiones

Este flujo permite registrar y consultar inversiones dentro de un portafolio.

1. El usuario autenticado selecciona un portafolio.
2. El sistema valida la identidad del usuario mediante JWT.
3. Se verifica que el usuario sea propietario del portafolio.
4. El usuario registra una inversión indicando criptomoneda y cantidad.
5. El sistema consulta el precio actual del activo mediante una API externa.
6. Se calcula el costo inicial de la inversión.
7. La inversión es almacenada en la base de datos.
8. Posteriormente, el usuario puede consultar sus inversiones asociadas al portafolio.

Este flujo integra información proveniente de servicios externos con los datos internos almacenados en el sistema.

4. Endpoints de la API

4.1. Autenticación

Método	Endpoint	Descripción
POST	/auth/register	Registra un nuevo usuario en el sistema.
POST	/auth/login	Inicia sesión con correo y contraseña.
POST	/auth/verify-mfa	Verifica el código MFA durante el inicio de sesión.
POST	/auth/setup-mfa	Genera la configuración inicial para MFA. Requiere autenticación.
POST	/auth/confirm-mfa	Confirma y activa MFA para el usuario. Requiere autenticación.
POST	/auth/forgot-password	Solicita recuperación de contraseña mediante correo electrónico.
POST	/auth/reset-password	Restablece la contraseña utilizando el token de recuperación.

4.2. Predicciones

Método	Endpoint	Descripción
POST	/predict	Genera una predicción basada en los datos enviados al servicio de ML.
POST	/predict/hour	Genera una predicción con horizonte temporal de una hora.

4.3. Portafolios

Método	Endpoint	Descripción
POST	/portafolios/create	Crea un nuevo portafolio asociado al usuario autenticado.
GET	/portafolios/list	Obtiene todos los portafolios pertenecientes al usuario autenticado.

4.4. Inversiones

Método	Endpoint	Descripción
POST	/inversiones/create	Registra una nueva inversión dentro de un portafolio.
GET	/inversiones/list	Obtiene todas las inversiones del usuario.
GET	/inversiones/id/:idInversion	Obtiene la información detallada de una inversión específica.

4.5. Modelos de Machine Learning (Backend Principal)

Método	Endpoint	Descripción
GET	/models	Obtiene todos los modelos registrados en el servicio de ML.
GET	/models/id/:id	Obtiene la información de un modelo específico por identificador.
GET	/models/symbol/:symbol	Obtiene los modelos asociados a un símbolo de criptomoneda.
GET	/models/active	Obtiene todos los modelos actualmente activos.
GET	/models/active/:symbol	Obtiene los modelos activos asociados a un símbolo específico.

5. Servicio de Machine Learning (Python)

Nota: Para información específica de qué enviar en cada request, se dispone de la documentación detallada de la API del servicio de Machine Learning en el repositorio de GitHub.

5.1. Entrenamiento de Modelos

Método	Endpoint	Descripción
POST	/models/train	Entrena un nuevo modelo para una criptomoneda específica. Uso administrativo.

5.2. Predicciones

Método	Endpoint	Descripción
POST	/models/predict	Genera predicciones usando el modelo activo de una criptomoneda.
POST	/models/predict/hour	Obtiene la predicción para una hora específica.

5.3. Consulta de Modelos

Método	Endpoint	Descripción
GET	/models/	Obtiene todos los modelos registrados.
GET	/models/id/{model_id}	Obtiene un modelo específico por identificador.
GET	/models/symbol/{symbol}	Obtiene los modelos asociados a una criptomoneda.
GET	/models/active	Obtiene todos los modelos actualmente activos.
GET	/models/active/{symbol}	Obtiene el modelo activo de una criptomoneda específica.

6. Testing (Calidad de Software)

Para garantizar la calidad del sistema, se implementaron pruebas unitarias tanto en el backend principal desarrollado en Node.js como en el servicio de Machine Learning desarrollado en Python. El objetivo principal de estas pruebas fue validar el comportamiento de las funciones críticas de negocio de forma aislada, verificando que respondieron correctamente ante escenarios exitosos, errores controlados y entradas inválidas.

6.1. Backend Node.js

En el backend se realizaron pruebas unitarias sobre los componentes centrales de la aplicación, incluyendo:

- Servicios de autenticación y validaciones.
- Gestión de portafolios e inversiones.
- Integración orientada a servicios con el módulo de Machine Learning.
- Manejo consistente de errores globales.

Las pruebas verifican que la lógica de negocio funcione correctamente sin necesidad de depender de servicios externos o de la base de datos mediante el uso estructurado de *mocks*.

6.2. Servicio de Machine Learning

En el servicio desarrollado en Python se realizaron pruebas unitarias sobre las funcionalidades principales relacionadas con:

- Entrenamiento de modelos matemáticos y algoritmos.
- Generación de predicciones cronológicas.
- Obtención de esquemas e inventario de modelos registrados.
- Validación adaptativa de parámetros de entrada y manejo de errores de procesamiento.

Estas pruebas permiten validar el correcto funcionamiento de los algoritmos y de los endpoints del servicio de forma independiente.

7. Auditoría de Seguridad

Se realizó una auditoría de seguridad sobre la aplicación bajo la modalidad *gray box* (caja gris), contando con acceso al código fuente y a la base de datos de producción, y evaluando además el sistema desplegado desde el exterior como lo haría un atacante. La evaluación se apoyó en el **OWASP Top 10** como marco de referencia y empleó herramientas como Postman para las pruebas sobre la API, DBeaver para la inspección de la base de datos, **npm audit** para el análisis de dependencias y la revisión manual del código fuente. La auditoría se concentró en el módulo de autenticación, el almacenamiento de datos sensibles, la configuración del despliegue y la comunicación entre cliente y servidor.

7.1. Controles Correctamente Implementados

Se verificó un conjunto sólido de controles de seguridad funcionando adecuadamente:

- Las contraseñas se almacenan como hash *bcrypt*, nunca en texto plano.
- Tras cinco intentos de inicio de sesión fallidos, la cuenta se bloquea temporalmente durante quince minutos (respuesta HTTP 423), mitigando los ataques de fuerza bruta.
- El sistema protege contra la enumeración de usuarios, devolviendo respuestas idénticas exista o no el correo consultado.
- La política de contraseñas exige una longitud mínima de ocho caracteres con mayúsculas, minúsculas, números y caracteres especiales.
- El uso de un ORM con consultas parametrizadas neutraliza los intentos de inyección SQL (*SQLi*).
- Los tokens de recuperación de contraseña son aleatorios, de un solo uso, se almacenan cifrados y expiran a los quince minutos.
- El secreto de MFA (TOTP) se almacena cifrado con *AES-256* en la base de datos.
- El análisis de dependencias con **npm audit** no reveló vulnerabilidades conocidas.

7.2. Hallazgos Identificados y Plan de Acción (SSL/TLS Remediación)

Se detectaron nueve hallazgos que requieren atención, clasificados por severidad:

- **Severidad Alta:** Ausencia de HTTPS en producción (las comunicaciones viajan sin cifrar), base de datos de producción accesible desde cualquier punto de Internet, y presencia de una credencial almacenada en texto plano en la base de datos.
- **Severidad Media:** Ausencia de cabeceras de seguridad HTTP, nombre de usuario almacenado sin cifrar, manejo inconsistente de tokens malformados (algunos módulos responden 500 en lugar de 401) y gestión manual de secretos en el despliegue.
- **Severidad Baja:** Política de CORS permisiva y divulgación de la tecnología del servidor mediante la cabecera **X-Powered-By**.

Como parte de la estrategia preventiva de remediación de infraestructura cloud, la mitigación de la ausencia de cifrado en tránsito de la API se estructuró mediante la siguiente hoja de ruta técnica:

1. Adquisición de un certificado criptográfico SSL/TLS de confianza pública mediante **AWS Certificate Manager (ACM)** asociado al dominio corporativo del servicio.
2. Despliegue de un **Application Load Balancer (ALB)** en las subredes públicas que actúe como terminación SSL/TLS frontal en el puerto 443, encargándose del descifrado asimétrico de los datos de la sesión.
3. Enrutamiento interno del tráfico limpio mediante HTTP hacia el puerto 3000 del servidor Ubuntu (EC2) de forma totalmente transparente para la aplicación.

8. Conclusión

La aplicación presenta una base de seguridad sólida en su lógica de autenticación. Los hallazgos de mayor severidad se concentran en la capa de despliegue e infraestructura (HTTPS, exposición de la base de datos y manejo de datos sensibles) más que en el código de la aplicación. Se recomienda corregir los tres hallazgos de severidad alta antes de considerar el sistema apto para manejar datos de usuarios reales.